HLS implementation of an autoencoder scheme

ELEC-Y541 - Reconfigurable architectures

Emmeran Colot

2026

1 Introduction

The objective of this report is to showcase the work done in an effort to implement an autoencoder (AE) architecture on an FPGA using high-level synthesis (HLS) tools. The model uses the MNIST dataset, consisting of 28x28 black and white pixels of handwritten numbers, and attempts to compress those images while minimizing the difference between the original image and the reconstructed one.

A trained model, written with PyTorch, was already given and only inference had to be run on the final implementation. The model first had to be translated in C++, which is explained in Sec. 2. The resulting implementation then has been tuned with different synthesis directives, optimizing either the performances in Sec. 3 or the area in Sec. 4.

2 C++ Implementation

2.1 Auto-encoder Model Structure

The model consists of 4 encryption layers and 4 decryption layers. Each layer consists of a linear part where the input is multiplied by a *weights* matrix, and then added to a *bias* vector. This linear operation is then followed by an activation function, which is a rectified linear unit (ReLU) for all layers except for the output layer.

Each layer can therefore be implemented by calling `Layer` (given below) with the right sizes and weights/biasses. It can be mathematically summarized as

$$y_i = \begin{cases} 0 & \text{if} \quad (\mathbf{W}_{(\mathbf{i},:)}\mathbf{u} + b_i) < 0 \\ \mathbf{W}_{(\mathbf{i},:)}\mathbf{u} + b_i & \text{otherwise,} \end{cases} \tag{1}$$

where $\mathbf{u}$ and $\mathbf{y}$ are respectively the input and output vector, $\mathbf{W}$ the weights matrix and $\mathbf{b}$ the bias vector.

2.2 Layer Specialization

A first issue that arose from the basic approach of a single function called for every layer is the dynamic number of iterations in the **L0** and **L1** loops, preventing loop pipelining as it would

```
void Layer(float layerInput[IN_SIZE],
            float layerOutput[OUT_SIZE],
            float weights[OUT_SIZE][IN_SIZE],
            float bias[OUT_SIZE]) {
    L0:for(int row = 0; row < OUT_SIZE; row++) {
        L1:for(int col = 0; col < IN_SIZE; col++) {
            layerOutput[row] = layerOutput[row] + weights[row][col] *
               layerInput[col];
        }
        layerOutput[row] = max(0.0f, layerOutput[row] + bias[row]); //
           ReLU
    }
}
```

need to have an undetermined number of stages. For this reason, each type of layer[1] has a specific function with fixed sizes.

2.3 Stream Model

For area optimization, FIFOs should be used whenever possible. To allow to have only FIFOs in a layer, each input and output must be read from / written to a single time and in a sequential order. For this reason, `Layer` has been written with a buffer for the input as it was the only variable being used not sequentially

```
void Layer(float layerInput[IN_SIZE],
            float layerOutput[OUT_SIZE],
            float weights[OUT_SIZE][IN_SIZE],
            float bias[OUT_SIZE]) {
    float in_buffer[IN_SIZE];
    float tmp;
    L0:for(int row = 0; row < OUT_SIZE; row++) {
        tmp = bias[row];  // Start with bias
        L1:for(int col = 0; col < IN_SIZE; col++) {
            if (row == 0) {
                // Save to buffer if first encounter
                in_buffer[col] = layerInput[col];
            }
            tmp = tmp + weights[row][col] * in_buffer[col];
        }
        layerOutput[row] = max(0.0f, tmp); // ReLU
    }
}
```

2.4 Autoencoder Code

Apart from the layers, the full code contains a first flattening operation, which converts the loaded image from a 28x28 matrix into a single vector of 784 elements and that removes the input mean and variance.

[1] a *type of layer* designates a layer with a given input and output layer size.

The opposite is applied at the output, reshaping it in a matrix. Those two functions are not shown here as they don't bring anything useful to the report.

3 Layer-wise Optimization

Before optimizing the whole model, a single layer was optimized using hls directives. This gave a more precise understanding of the impact of the used directives.

3.1 Baseline

The baseline was taken by disabling pipelining and loop flattening on both **L0** and **L1** loops. It used the code introduced in Sec. 2.2.

3.2 Performance Oriented

3.2.1 Inner Loop Pipelining

By pipelining the loop **L1**, each iteration tries to be completed in a single cycle (II = 1 by default) to process the next iteration right afterwards.
There is however a feedback in the loop as the output of the layer is updated for each row of the input. The math being run in floating point, the addition takes multiple cycles and the resulting design can only assure an initiation interval (II) of 3 cycles, as shown on Fig. 1.

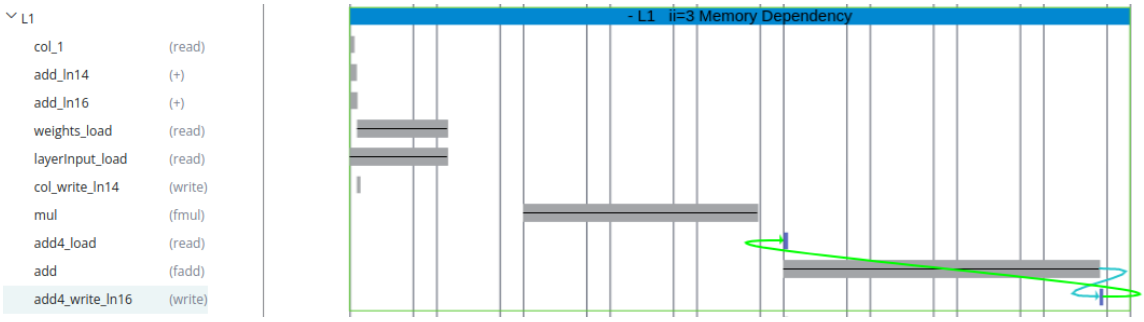

Figure 1: Schedule of the inner loop when pipelined **L1**.

An iteration of the whole loop taking 9 cycles, the pipelining should result in a division of the interval by 3. As the outer loop is not modified, the latency should be improved by the same factor and stay similar to the interval.

3.2.2 Outer Loop Pipelining

L0 can be pipelined in order to reduce the interval. This will unroll the inner loop as a pipelined loop must not contain any other loop. The function will now compute a new row of the output at each cycle, making the interval drop to OUT_SIZE[2].
Because **L1** is unrolled, each of its iterations is performed simultaneously. This means *weights* and *layerInput* should have multiple read operations happening at the same cycle. To do so,

[2]equal to 256 here

these arrays are partitioned along the accessed dimension, which is the first one for the input layer and the second one for the weights matrix.
The resulting latency can be computed as the number of pipeline stages, to which the number of cycles of a single stage must be added. Fig. 2 gives an insight on how the inner loop has been flattened. As the operation is a multiplication followed by a sum, each input row is multiplied with the corresponding weight and is then added to the sum computed by the previous input row.

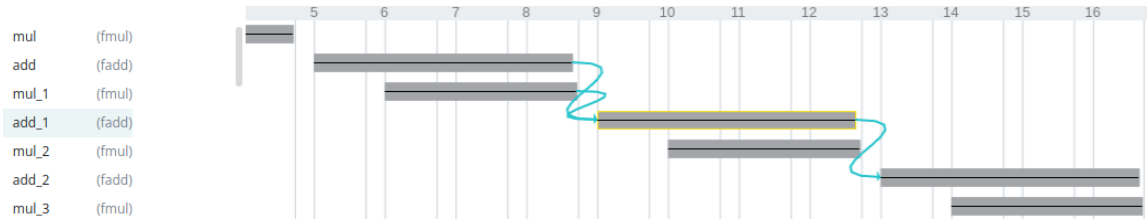

Figure 2: Partial schedule of the pipelined outer loop.

This addition takes 4 cycles and it must be applied for each input, so it is repeated IN_SIZE[3] times. This gives an approximate latency cycle count of $256 + 4 \times 256 = 1280$.

A quick note here: knowing the output is simply the sum of the effect of each input and the ordering of the sum has no effect, the latency could be reduced by summing the input contributions in a "tree-like" manner instead of a "chain-like" summation. The HLS tool is probably capable of such things but it goes outside of the scope of this course.

3.3 Area Oriented

For area optimization, the sequential code of Sec. 2.3 has been used to take advantage of the low resource consumption of FIFOs. To avoid duplicating computing elements, loop unrolling and pipelining have been disabled, together with the partitioning as parallel access is no longer needed.
The use of FIFOs removes the need for some flip flops (FF) where data does not need to be accessed multiple times, which makes it use less area.

3.4 Results

The 4 versions, differing only in the synthesis directives for the first 3 and with a slight code modification for the last one, were synthesized and the result is shown on Fig. 3.

[3] equal to 256 too

	Layer_0_Basic	Layer_1_InnerPipelining	Layer_2_OuterPipelining	Layer_3_FIFO
Component Information (5)				
Component Metrics (3)				
C SYNTHESIS (11)	✓	✓	✓	✓
Clock Target (ns)	10	10	10	10
Clock Uncertainty (ns)	2.7	2.7	2.7	2.7
Clock Achieved (ns)	7.016	7.016	7.016	7.016
Interval (cycles)	592386	200706	256	656130
Latency (cycles)	592385	200705	1291	656129
Latency (ns)	5924000	2007000.0000000002	12910	6561000
BRAM	0	0	0	1
DSP	5	5	1282	5
FF	635	642	167257	542
LUT	675	723	107551	660

Figure 3: Result of the different versions for a single layer.

The interval and latency for the inner and outer pipelining have already been explained and the last design has, without any surprise, worse performance metrics as they were traded for less components usage. The gain is mostly visible in the FF count.
Concerning the resources usage of the inner pipelining design, it only gains a few FFs and LUTs because only a small part of the code is pipelined. The outer pipelined design sees a major increase as the inner loop is unrolled, needing 256 times more resources[4].
Table 1 gives the improvement of each design with regards to the baseline and clearly show how much the outer pipelined design improved performance metrics and the FIFO design improved the area metrics.

Table 1: Performance Improvement Relative to Baseline

	Inner Pipelining	**Outer Pipelining**	**FIFOs**
Interval	33.88%	0.04%	110.76%
Latency	33.88%	0.22%	110.76%
DSP	100.00%	256.40%	100.00%
FF	101.10%	263.40%	85.35%
LUT	107.11%	159.33%	97.77%

4 AE Optimization

[4] $256 \times 642 = 164.352$ being quite close to the true FF count.